

Report of SUSTech Campus RAG

Campus Knowledge via RAG

Gao Fei

Zhang Sihua

Gao Ruilin

Large AI Models

Artificial Intelligence

Southern University of Science and Technology

June 14, 2026

Abstract

This project presents a local-first Retrieval-Augmented Generation (RAG) system for querying publicly available web content from the Southern University of Science and Technology (SUSTech). The system integrates web data acquisition, text preprocessing, semantic chunking, vector-based retrieval, reranking, and large language model (LLM) generation into a unified end-to-end pipeline for natural language question answering over campus knowledge.

The backend is implemented in Python with FastAPI, supporting configurable LLM inference backends including llama.cpp and vLLM. Semantic retrieval is powered by BGE embedding and reranking models, and response generation is handled by Qwen3-based models. The frontend is developed with Vue 3 and Vite, delivering a responsive web interface with real-time streaming responses via Server-Sent Events (SSE), multi-session management, and customizable UI themes.

The system is designed to be modular and deployment-flexible, enabling operation on local workstations and GPU-enabled servers alike. By combining efficient dense retrieval with generative language models, the system substantially improves the accessibility and usability of campus information.

Contents

	Page
1 Introduction	4
1.1 Motivation	4
1.2 System Overview	5
1.3 Report Organisation	5
2 Background	6
2.1 Retrieval-Augmented Generation	6
2.2 Self-RAG	6
2.3 Hybrid Retrieval	7
3 Data Ingestion Pipeline	8
3.1 Web Crawling	8
3.2 Text Preprocessing and Quality Filtering	9
3.3 Semantic Chunking Strategy	10
4 Retrieval and Reranking	11
4.1 Dense Retrieval with BGE Embeddings	11
4.2 Sparse Retrieval with BM25	11
4.3 Reciprocal Rank Fusion	12
4.4 Cross-Encoder Reranking	12
5 Self-RAG Reasoning Loop	14
5.1 Scope Detection	14
5.2 Per-Candidate Relevance Assessment	14
5.3 Factual Support Validation	16
5.4 Multi-Round Retrieval and Fallback	16
5.5 Robustness of Structured Output Parsing	16
5.6 Simple RAG Mode	17
6 System Integration	18
6.1 LLM Backend Abstraction	18
6.2 llama.cpp Backend	18

6.3	vLLM Backend	18
6.4	Frontend Interface	19
6.5	Relay-Worker Deployment	19
7	Limitations and Future Work	20
7.1	Absence of a Quantitative Evaluation	20
7.2	Retrieval Quality Cannot Be Assessed	20
7.3	Self-RAG Effectiveness is Unverified	21
7.4	Data Coverage and Freshness	21
7.5	Future Directions	21
8	Conclusion	23
	References	24

1 Introduction

1.1 Motivation

University websites are the primary channel through which institutions communicate academic, administrative, and campus-life information to students and staff. At the Southern University of Science and Technology (SUSTech), this content is distributed across hundreds of web pages covering departmental announcements, administrative procedures, facility information, and event notices. Traditional keyword search returns a ranked list of pages and leaves the user to read and synthesise the answer themselves. This places the burden of information extraction on the user and produces poor results for questions that span multiple pages or require understanding natural language context rather than matching terms.

Large language models (LLMs) have demonstrated the ability to understand and generate fluent natural language across a wide range of tasks. However, a general-purpose LLM trained on internet-scale data has no reliable knowledge of institution-specific details—current staff contacts, procedural requirements, room assignments, course announcements—that change frequently and were either absent from or underrepresented in training corpora. Querying such a model directly produces responses that are fluent but factually unreliable.

Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) addresses this gap by combining the generative capability of LLMs with the factual grounding of document retrieval. Rather than relying on parametric memory, a RAG system retrieves relevant passages from a curated knowledge base at query time and conditions generation on that evidence. This makes the system’s knowledge updatable by re-indexing documents, traceable through source citations, and controllable through the scope of the indexed corpus.

This project builds a local-first RAG system for SUSTech’s public web content. “Local-first” means the entire pipeline—embedding, retrieval, reranking, and language model inference—runs on local hardware without reliance on external APIs, ensuring data privacy and full operational control. The system is designed to handle the practical challenges of real-world campus content: mixed Chinese and English text, structured announcement templates, short informational pages, and a heterogeneous crawl of several hundred pages from across the SUSTech website.

1.2 System Overview

The system consists of two independent phases. In the *offline indexing phase*, publicly available pages are crawled from the SUSTech website, cleaned to remove navigation boilerplate and low-quality content, segmented into semantically coherent chunks, and encoded as dense vectors using BGE embedding models (Chen et al., 2024). The resulting chunks and their embeddings are persisted in a ChromaDB (Chroma Team, 2023) vector store alongside a BM25 index for sparse retrieval.

In the *online inference phase*, a user query triggers a multi-stage pipeline. Hybrid retrieval combines dense nearest-neighbour search with BM25 sparse retrieval, fused via Reciprocal Rank Fusion (RRF) (Cormack, Clarke, and Buettcher, 2009). A cross-encoder reranker refines the candidate set. A self-RAG reasoning loop (Asai et al., 2024) then uses the language model itself to assess whether the query is in scope, whether each retrieved chunk is genuinely relevant, and whether the generated response is factually supported by the evidence. Generation is handled by Qwen3 (Qwen Team, 2025), running on either llama.cpp (Gerganov, 2023) or vLLM (Kwon et al., 2023) depending on the deployment environment. The response is streamed token-by-token to a Vue 3 frontend interface via Server-Sent Events.

1.3 Report Organisation

Chapter 2 provides background on RAG, Self-RAG, and hybrid retrieval. Chapter 3 describes the data ingestion pipeline: crawling, preprocessing, and chunking. Chapter 4 covers the hybrid retrieval and reranking architecture. Chapter 5 explains the self-RAG reasoning loop in detail. Chapter 6 discusses system integration: the LLM backend abstraction, frontend interface, and relay-worker distributed deployment. Chapter 7 critically examines the system’s limitations, particularly the absence of quantitative evaluation. Chapter 8 concludes.

2 Background

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation was introduced by Lewis et al. (2020) as a method for grounding language model outputs in external document collections. A retriever—typically a dense bi-encoder—embeds queries and documents into a shared vector space and retrieves the most similar documents by nearest-neighbour search. A generator then conditions on both the query and the retrieved documents to produce the final answer. The key advantage over purely parametric approaches is that the knowledge base can be updated by re-indexing documents without retraining the model, and retrieved passages provide an auditable evidence trail for generated claims.

The practical significance of RAG for domain-specific question answering is substantial. A general-purpose LLM trained on internet-scale data has no reliable knowledge of institution-specific details—current staff contacts, procedural requirements, room assignments—that change frequently and were not prominently represented in training corpora. RAG provides a principled mechanism to inject this dynamic, local knowledge at inference time.

2.2 Self-RAG

A notable limitation of standard RAG is that the system blindly generates a response regardless of whether the retrieved passages are actually relevant or sufficient to answer the question. Asai et al. (2024) address this with Self-RAG, a framework where the model generates special *reflection tokens* to decide when retrieval is needed, assess the relevance of each retrieved passage, and evaluate whether its own output is supported by the evidence. In the original formulation, the model is fine-tuned end-to-end to produce these tokens as part of the standard generation process.

Our implementation adopts the spirit of Self-RAG without task-specific fine-tuning. A general-purpose Qwen3 model is prompted to perform each reflection step as a structured classification task, producing JSON-formatted judgements that control the pipeline’s branching logic. This trades the elegance of the original single-model approach for deployment simplicity: no specialised training data or fine-tuning infrastructure is required.

2.3 Hybrid Retrieval

Dense retrieval using bi-encoder models captures semantic similarity: query and document are encoded independently and compared by inner product or cosine similarity. This works well when the query and relevant passage use different vocabulary for the same concept. However, dense models can fail on exact-match queries—specific names, course codes, or building identifiers—if these terms are not well-represented in the embedding space.

Sparse retrieval methods such as BM25 (Robertson and Zaragoza, 2009) rely on lexical overlap and inverse document frequency weighting, excelling precisely where dense retrieval struggles. The complementary nature of these two signals makes their combination consistently stronger than either alone. Reciprocal Rank Fusion (RRF) (Cormack, Clarke, and Buettcher, 2009) provides a simple, parameter-free method for merging two ranked lists: results are re-ranked by summing reciprocal rank scores, requiring no normalisation of heterogeneous similarity scores.

Once a candidate set is assembled through retrieval, a cross-encoder reranker (Reimers and Gurevych, 2019) jointly encodes the query and each candidate passage, enabling full attention-based interaction modelling. Cross-encoders are substantially more accurate than bi-encoders at judging fine-grained relevance, but are too expensive to apply to the entire corpus; they are therefore used only to re-rank a small pre-filtered candidate set.

3 Data Ingestion Pipeline

Before retrieval can occur, the raw web content of the SUSTech official website must be transformed into a clean, indexed corpus. Figure 3.1 illustrates the complete end-to-end system pipeline; this chapter focuses on the three offline stages: web crawling, text preprocessing and quality filtering, and semantic chunking.

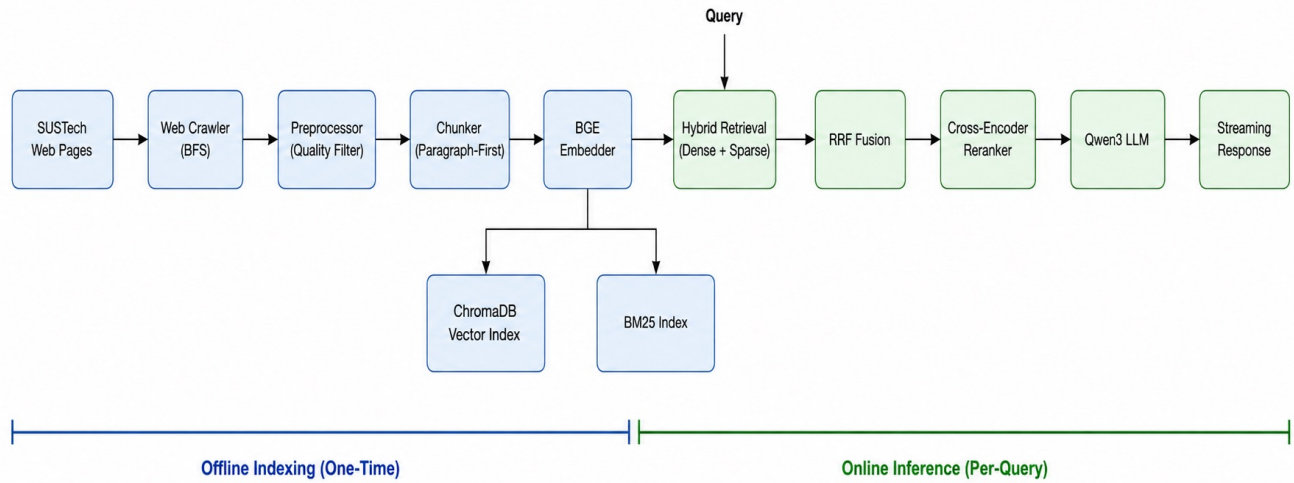


Figure 3.1: End-to-end RAG pipeline. Blue boxes represent the offline indexing phase; green boxes represent the online inference phase.

3.1 Web Crawling

The crawler performs a breadth-first search (BFS) traversal starting from a configurable set of seed URLs. At each step it fetches the current page, extracts outgoing links, and enqueues those that (a) belong to the same domain, (b) do not resolve to non-text resources (.jpg, .zip, .doc, and similar extensions), and (c) have not already been visited. Traversal halts when the queue is empty or the page limit is reached (default: 200 pages).

URL normalisation is applied before deduplication. Query parameters associated with analytics and tracking (utm_*, spm, source, _t, _refluxos) are stripped, fragment identifiers are removed, and

duplicate query keys are collapsed. Without this step, the same content page would be indexed multiple times under different tracking-annotated URLs, inflating the corpus and skewing retrieval results.

Content extraction uses a two-stage approach. The primary extractor applies `readability-lxml`, which identifies the main article body using heuristics similar to browser reader modes. When readability fails or returns empty output, a fallback extractor uses BeautifulSoup to prune the DOM of non-content elements (navigation bars, scripts, stylesheets, footers, sidebars, iframes) and select the most content-rich remaining subtree. Each page’s raw HTML and extracted text are persisted to disk alongside its source URL for provenance tracking.

3.2 Text Preprocessing and Quality Filtering

Raw extracted text contains substantial noise: navigation remnants, copyright footers, repeated boilerplate, and pages that are effectively empty after extraction. The preprocessing stage applies a series of filters at both the line and document level.

Line-level cleaning. Lines matching known boilerplate patterns—copyright notices, ICP filing identifiers, and similar administrative footers—are dropped. Three or more consecutive blank lines are collapsed to two. A broader set of boilerplate markers (phone numbers, postal codes, QR-code mentions, login prompts) is used to identify lines as non-substantive without discarding them from the raw output; their prevalence is instead used in the document-level quality score.

Document-level quality gating. A document is accepted only if it satisfies three criteria simultaneously:

1. Its total character count exceeds a minimum threshold of 60 characters.
2. The proportion of repeated lines—lines of eight or more characters that appear more than once, excluding pure dates and numeric sequences—does not exceed 35%.
3. It contains at least one *substantive* line: either a line of 24 or more characters, or one containing Chinese sentence-ending punctuation (full stop, exclamation mark, question mark, or semicolon in full-width form).

Documents that fail any criterion are discarded before chunking. The thresholds are intentionally conservative: false negatives (keeping noisy documents) are more harmful to downstream retrieval quality than false positives (discarding borderline content), since a noisy chunk occupying a retrieval slot can directly cause a confusing or incorrect response.

3.3 Semantic Chunking Strategy

Splitting documents into chunks of a size suitable for dense retrieval while preserving semantic coherence is a genuinely non-trivial problem. A naive fixed-window approach that splits every 500 characters without regard for sentence or paragraph structure routinely cuts sentences mid-thought, creating chunks that are internally incomplete and misleading to a retriever.

Our chunker uses a *paragraph-first* strategy. Documents are first split at double-newline boundaries into paragraphs. Short paragraphs are accumulated in a running buffer until the combined length approaches the target chunk size (default: 500 characters), then flushed as a single chunk. Paragraphs that individually exceed the target size are processed by a sentence-level splitter (`SentenceSplitter` from `LlamaIndex`), configured with a 100-character overlap between consecutive sentence-level chunks.

Two special cases are handled explicitly:

Structured announcement pages. Many campus pages follow a short template: a title followed by rows of key-value pairs (e.g. “Time: 2025-09-01”, “Location: Building 1”, “Contact: Ms. Li”). Splitting such pages at sentence boundaries destroys the table-like structure and renders individual chunks meaningless out of context. The chunker detects these pages using three criteria: at least three lines, total content under 750 characters, and at least 50% of lines matching structural patterns (colon separators, Chinese field-name keywords, or date formats). Pages satisfying all three criteria are preserved as a single chunk regardless of length.

Short chunk merging. After the initial split, chunks shorter than $\max(120, \text{chunk_size}/3)$ characters are merged with an adjacent neighbour, provided the merged result stays within $1.35 \times \text{chunk_size}$ characters. This post-processing step eliminates sparse “island” chunks—typically trailing sentence fragments at the end of a section—that would occupy retrieval slots without providing useful context.

Each chunk retains metadata linking it to its source URL, page title, document identifier, and the content extractor (readability or BeautifulSoup fallback) that produced the original page. This provenance information is used to generate source citations in the frontend interface and to support debugging of pipeline failures.

4 Retrieval and Reranking

Given an indexed corpus of chunks, the retrieval stage must identify those most likely to contain information relevant to a user’s query. This chapter describes the two complementary retrieval signals the system uses, how they are combined, and the reranking step that selects the final context passed to the language model.

4.1 Dense Retrieval with BGE Embeddings

Dense retrieval encodes both queries and chunks as fixed-dimensional vectors and finds relevant chunks by nearest-neighbour search in that vector space. Because the model encodes semantic content rather than surface lexical form, it can retrieve passages that are topically relevant even when their vocabulary diverges from the query.

The system uses BGE (*BAAI General Embedding*) models (Chen et al., 2024), with a choice of model scaled to the deployment environment: BAAI/bge-small-zh-v1.5 for lightweight local inference and Qwen/Qwen3-Embedding-4B (bfloat16, batch size 4, cuda:2) for the production GPU server. Both models have strong performance on Chinese text, which comprises the majority of the SUSTech website’s content. Embeddings are stored persistently in ChromaDB (Chroma Team, 2023). At query time, the query is embedded with the same model and used to retrieve the top k chunks by cosine similarity ($k = 8$ by default).

4.2 Sparse Retrieval with BM25

Dense retrieval can fail on queries that require exact keyword matching—specific course codes, building names, or proper nouns that may not cluster closely to their descriptions in the embedding space. BM25 (Robertson and Zaragoza, 2009) addresses this limitation through a term-frequency and inverse-document-frequency scoring scheme that rewards documents containing the query’s exact terms, particularly rare ones.

The system maintains a BM25 index built from `chunks.jsonl` at indexing time. At query time, sparse retrieval independently returns a top- k candidate list ($k = 8$ by default). This list may overlap with or be entirely disjoint from the dense retrieval results, depending on the nature of the query.

4.3 Reciprocal Rank Fusion

Dense and sparse candidate lists are merged using Reciprocal Rank Fusion (RRF) (Cormack, Clarke, and Buettcher, 2009). For a chunk d appearing at position $\text{pos}_r(d)$ in ranked list r , its RRF score is defined as:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{pos}_r(d)} \quad (4.1)$$

where R is the set of rankers (dense and sparse) and $k = 60$ is a smoothing constant. Chunks absent from a ranker’s list contribute zero from that term. The merged list is sorted by decreasing RRF score and passed to the reranker.

RRF is attractive precisely because it requires no score-level normalisation. The cosine similarity scores produced by BGE and the BM25 term-frequency scores are incommensurable and cannot be combined directly. By operating on ranks rather than scores, RRF treats both signals symmetrically without any calibration. The value $k = 60$ follows Cormack, Clarke, and Buettcher (2009), who found it robust across multiple retrieval benchmarks. Dense and sparse retrieval are executed in parallel using a thread pool to minimise added latency.

4.4 Cross-Encoder Reranking

The RRF-fused list may still contain passages that are topically adjacent to the query but do not actually contain the sought information. Bi-encoder models, used for dense retrieval, encode query and document independently, which limits their ability to model fine-grained query-passage interaction. Cross-encoder models (Reimers and Gurevych, 2019) concatenate query and passage as a single input and score them through full self-attention, enabling richer interaction at the cost of higher per-candidate compute. They cannot be applied to the full corpus (where latency would be prohibitive) but are well-suited to re-scoring a small pre-filtered candidate set.

The reranker uses BGE cross-encoder models (BAAI/bge-reranker-v2-m3 in the default configuration, Qwen/Qwen3-Reranker-4B in production, placed on cuda:2). It scores each RRF-fused candidate against the query and returns the top $n = 4$ chunks by reranker score. These four chunks form the context passed to the self-RAG reasoning loop or, in simple RAG mode, directly to the language model for generation.

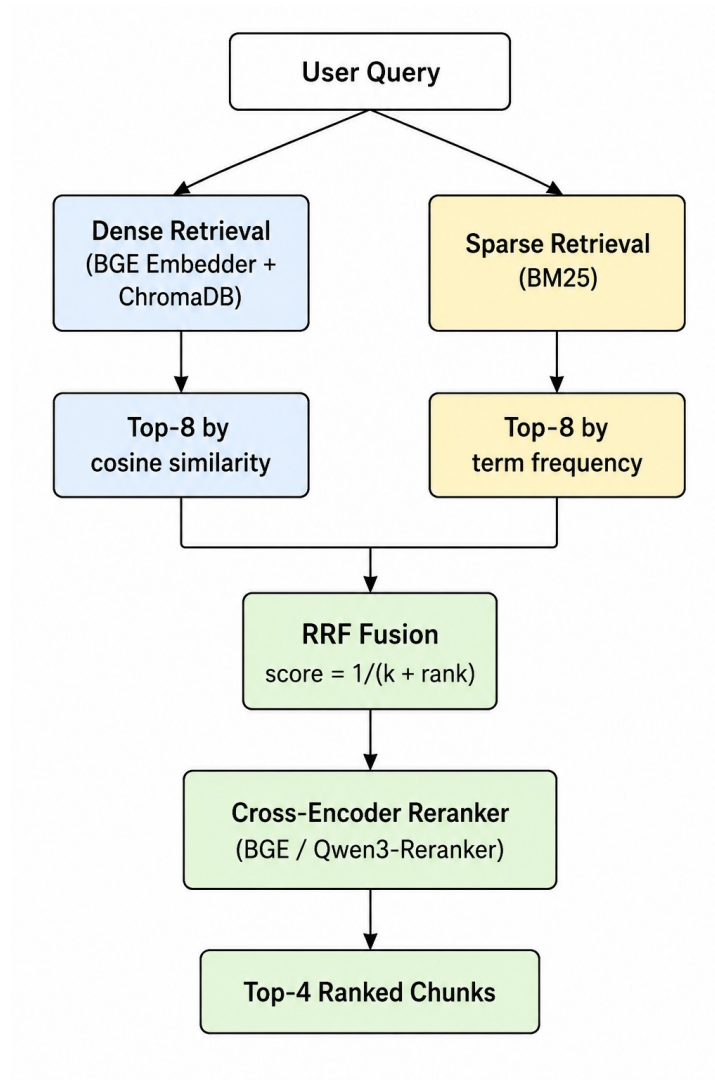


Figure 4.1: Hybrid retrieval and reranking architecture. Dense and sparse retrieval run in parallel; their ranked lists are merged by RRF and re-scored by a cross-encoder.

5 Self-RAG Reasoning Loop

A standard RAG pipeline retrieves the top- k chunks and unconditionally feeds them to the language model regardless of whether they actually address the query, or whether the generated response is factually grounded in the evidence. In a closed-domain system like ours—where the knowledge base covers only SUSTech public web content—this creates two distinct failure modes. First, the model may confidently answer an out-of-scope question by drawing on its parametric knowledge, producing a plausible but unverifiable response. Second, retrieved passages may score well on embedding similarity without containing information directly relevant to the specific question asked, leading to responses that are confidently stated but factually unsupported.

Our system addresses both failure modes through a multi-stage reasoning loop inspired by Self-RAG (Asai et al., 2024). Rather than fine-tuning the model to emit reflection tokens, we prompt the same Qwen3 model to perform structured classification at each decision point, producing JSON-formatted judgements that drive pipeline branching. The complete loop is illustrated in Figure 5.1.

5.1 Scope Detection

Before any retrieval is triggered, the LLM is asked whether the query falls within the scope of the SUSTech knowledge base. The model is given a system prompt framing it as a routing classifier: it should output `{"should_retrieve": true}` for factual questions about SUSTech—departments, administrative procedures, campus events, facilities, and personnel—and `{"should_retrieve": false}` for clearly out-of-scope requests (general knowledge questions, external information, or casual conversation). When retrieval is not warranted, the system returns a pre-formulated, polite out-of-scope response without invoking the retrieval stack or the generation model, avoiding unnecessary latency and eliminating a class of hallucination entirely.

5.2 Per-Candidate Relevance Assessment

After retrieval and reranking return up to four candidate chunks, each chunk is individually evaluated for relevance to the specific query. The LLM receives the query and the text of each candidate and returns a structured list of relevance judgements:

```
[{"candidate_index": 0, "relevant": true, "reason": "..."},  
 {"candidate_index": 1, "relevant": false, "reason": "..."}]
```

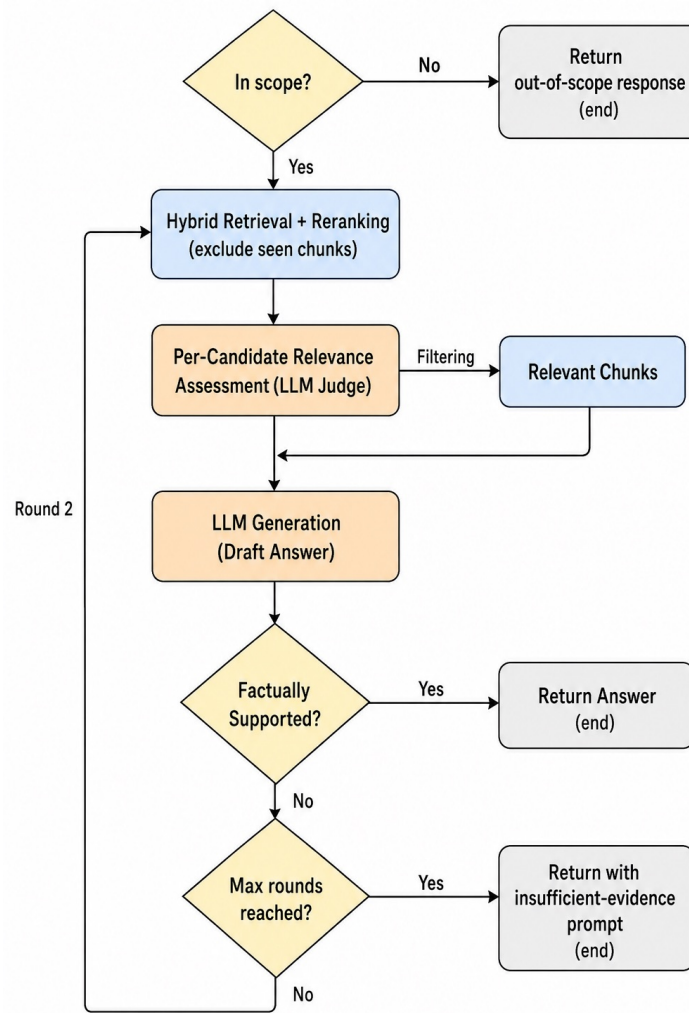


Figure 5.1: Self-RAG reasoning loop. Orange boxes involve LLM inference calls; grey boxes are terminal states. The loop runs for at most two retrieval rounds before falling back to an insufficient-evidence response.

The system prompt instructs the model to apply a strict threshold: a chunk must contain information that *directly* addresses the query, not merely mention related entities or keywords. Chunks whose only connection to the query is topical proximity—sharing a subject domain but not providing the specific fact sought—are marked irrelevant. Only chunks passing this filter proceed to generation.

This step is complementary to, not a replacement for, the cross-encoder reranking already applied. Reranking orders the candidate set by relative relevance but cannot determine that the *entire* retrieved set is tangentially related to the query. The LLM judge introduces an absolute relevance threshold that can reject all candidates if none are genuinely useful.

5.3 Factual Support Validation

After the LLM generates a draft answer from the filtered context, the system validates whether the answer is factually *supported* by the retrieved evidence. A separate LLM call presents the draft answer alongside all retrieved passages and asks the model to return:

```
{"supported": true/false, "reason": "..."} 
```

The system prompt defines support strictly: every factual claim in the answer must be *explicitly stated* in the retrieved text. Inference, extrapolation, and commonsense completion—even plausible ones—count as unsupported. This is intentionally conservative: the goal is to identify cases where the model has gone beyond the evidence, not to validate answers that are merely consistent with it.

5.4 Multi-Round Retrieval and Fallback

If the support check fails in the first round, the pipeline does not immediately return the unsupported answer. Instead, it initiates a second retrieval round, this time excluding all chunks already assessed in round 1. The intuition is that the first retrieval may have saturated on the most superficially similar passages; by excluding them, the second round is forced to explore a different region of the corpus that might supply the missing evidence.

The loop runs for at most `max_rounds=2` iterations. At the end of each round, relevant chunks are accumulated across rounds, so the second round’s context builds on—rather than replaces—evidence found in the first. If after all rounds the evidence still does not support a factual answer, the model generates a response under a system prompt that explicitly instructs it to acknowledge insufficient evidence rather than fabricate. If no relevant chunks are found at all, a separate prompt communicates that the query is within scope but currently unanswerable from the indexed corpus.

5.5 Robustness of Structured Output Parsing

Each LLM-in-the-loop decision point requires the model to produce valid JSON. In practice, language models occasionally emit malformed output: trailing commas, truncated responses due to token limits, or extraneous text surrounding the JSON object. The system uses a two-tier parsing strategy to handle this:

1. **Constrained generation.** If the LLM backend supports JSON schema enforcement—llama.cpp via GBNF grammar, or vLLM via structured decoding—the model is constrained to emit only tokens matching the required schema. This eliminates malformed output at the source.

2. **Repair fallback.** If constrained generation is unavailable or fails, the system extracts the first JSON-like substring from the raw output and applies a repair pass: it strips leading/trailing non-JSON text, removes trailing commas before closing delimiters, and closes any uncapped braces, brackets, or string quotes by traversing the token stream as a FILO stack.

This two-tier approach keeps the self-RAG loop functional across all supported backends and quantization configurations, at the cost of additional parsing logic in the fallback path.

5.6 Simple RAG Mode

For latency-sensitive deployments where the overhead of multiple LLM calls is unacceptable, the system provides a simple RAG mode (`retrieval.mode: "simple"`) that bypasses all reflection steps. In this mode, retrieval, reranking, and generation proceed in a single pass with no branching. The mode is switched via the configuration YAML without modifying application code. This design enables direct comparison of simple and self-RAG behaviour; however, as discussed in Chapter 7, no quantitative comparison has been conducted.

6 System Integration

6.1 LLM Backend Abstraction

The generation layer is implemented behind a common interface with two concrete backends, switchable via the `llm.backend` field in the configuration YAML. Both backends expose the same methods: `generate(messages)` for single-turn completion, `generate_stream(messages, cancel_event)` for token-by-token streaming, and optionally `generate_with_schema(messages, schema)` for constrained JSON output. This abstraction makes the rest of the pipeline—including the self-RAG loop—entirely backend-agnostic.

Streaming output uses an event model that distinguishes *thinking* tokens (Qwen3’s chain-of-thought reasoning, emitted before the visible response) from *content* tokens (the final answer). The frontend renders these differently: thinking tokens appear in a collapsible section, while content tokens accumulate in the main chat bubble. A `threading.Event` cancellation handle is threaded through the pipeline to allow in-flight requests to be aborted mid-generation when the user navigates away or clears the conversation.

6.2 llama.cpp Backend

The llama.cpp backend (Gerganov, 2023) manages a `llama-server` subprocess that loads a GGUF-quantized model file. The default deployment uses `Qwen3-8B-Q4_K_M.gguf`, a 4-bit quantized variant of Qwen3-8B. Key inference parameters include a context window of 8192 tokens, temperature 0.2, `top-p=0.95`, KV-cache quantisation (`q8_0`) to reduce VRAM footprint, and a batch size of 512. This backend targets CPU or single-GPU machines and is configured for one concurrent request, trading throughput for simplicity.

6.3 vLLM Backend

The vLLM backend (Kwon et al., 2023) manages a `vllm serve` subprocess for the production environment ($3 \times$ RTX 4090). It loads `Qwen3-30B-A3B-Instruct-AWQ`, a 30-billion-parameter mixture-of-experts model with AWQ weight quantisation. Tensor parallelism of degree 2 distributes the model across `cuda:0` and `cuda:1`, while the embedding and reranking models are isolated on `cuda:2` to avoid

GPU memory contention. With 88% GPU memory utilisation, prefix caching, and chunked prefill enabled, the backend supports up to 32 concurrent requests. vLLM’s PagedAttention memory management avoids the memory waste of static KV-cache allocation, substantially improving throughput on bursty multi-user workloads.

6.4 Frontend Interface

The frontend is a Vue 3 and Vite application that communicates with the backend via a RESTful API and receives streaming responses through Server-Sent Events (SSE). The SSE protocol carries ten event types; the most significant for the user experience are `reference` (retrieved source URLs displayed as footnote citations), `think.delta` and `think.end` (Qwen3 reasoning stream), and `content.delta` (response token stream). Multi-session management and user preferences are persisted in browser `localStorage`. The application adapts its layout automatically between desktop and mobile viewports, and also ships embedded and floating-widget variants for integration into third-party pages.

6.5 Relay-Worker Deployment

For deployments where the GPU machine is behind a NAT or firewall and cannot accept inbound connections, the system provides a relay-worker architecture. A stateless relay process runs on a cloud host with a public IP; it performs no inference and requires minimal compute. GPU workers run on the local machine and establish an outbound WebSocket connection to the relay, registering their capabilities (model name, available VRAM, and empirically measured tokens per second) on connect.

When a query arrives at the relay via HTTP, the relay selects the idle worker with the highest self-reported throughput and forwards the task as a WebSocket message. The worker executes the full RAG pipeline and streams events back to the relay, which re-serialises them as SSE and forwards them to the waiting client. A heartbeat mechanism (30-second interval, 90-second timeout) removes silently disconnected workers. Workers automatically reconnect after a 30-second delay on disconnect.

This architecture decouples API availability from GPU hardware management: the relay can be deployed on a low-cost cloud instance, while GPU workers run wherever the hardware happens to be. Multiple workers can register behind a single relay, with the relay routing new requests to the fastest idle worker.

7 Limitations and Future Work

7.1 Absence of a Quantitative Evaluation

The most consequential limitation of this project is the absence of a labelled evaluation dataset for the SUSTech domain. Without a collection of (question, reference answer, relevant passages) triples drawn from real campus information needs, it is impossible to measure any aspect of the system’s performance quantitatively: not retrieval precision or recall at any rank cutoff, not the faithfulness of generated answers to the retrieved evidence, not end-to-end answer accuracy, and not the marginal contribution of any individual pipeline component.

This matters because the system contains numerous design choices—hybrid retrieval versus dense-only, self-RAG versus simple RAG, specific values of chunk size, top- k , and temperature—that are motivated by reasoning and prior work in the retrieval-augmented generation literature but have not been empirically validated on *this* corpus and *this* query distribution. A choice that is correct in expectation over many benchmark datasets may be poorly calibrated for the specific characteristics of Chinese university web content.

Standard RAG evaluation frameworks decompose system quality into dimensions such as context precision (what fraction of retrieved chunks is relevant?), context recall (what fraction of relevant information is retrieved?), faithfulness (is the generated answer entailed by the retrieved context?), and answer relevance (does the answer actually address the question?). Measuring these on even a modest benchmark of 100–200 questions would transform the system from a functioning prototype into a system with known, bounded properties. Constructing such a benchmark—through manual annotation by campus stakeholders or semi-automatic LLM-based question generation followed by human verification—is the single highest-priority item for future work.

7.2 Retrieval Quality Cannot Be Assessed

The hybrid retrieval architecture (dense + sparse + RRF + cross-encoder reranking) is substantially more complex than a dense-only baseline. Complexity is justified only if it improves retrieval quality, and that improvement has not been measured.

Specifically, we do not know whether BM25 sparse retrieval adds information not already captured by BGE dense retrieval on this corpus; whether RRF fusion outperforms either individual ranker alone; or whether the cross-encoder reranker meaningfully reorders the RRF-fused list relative to leaving

the RRF ranking unchanged. On a domain-specific Chinese corpus composed largely of short, structured documents, it is plausible that dense retrieval alone would perform comparably to the full hybrid pipeline at lower latency. The only way to resolve this is through ablation experiments over a labelled evaluation set.

7.3 Self-RAG Effectiveness is Unverified

The self-RAG reasoning loop introduces between two and six additional LLM calls per query (scope detection, per-candidate relevance assessment for up to four candidates, support validation, and potentially a second retrieval round). On the llama.cpp backend with a single concurrent request, each call can take several seconds, substantially increasing end-to-end latency compared to simple RAG mode.

Whether this cost is justified by improved answer quality is an open question. The scope detection step may reject legitimate in-scope queries if the classifier is miscalibrated. The per-candidate relevance filter may discard passages that are genuinely useful but indirectly phrased. The support validation step may trigger unnecessary re-retrieval when the answer is correct but worded differently from its source text. All of these are failure modes of the self-RAG approach that would surface under systematic evaluation but are invisible without it. The `retrieval.mode: "simple"` configuration switch was precisely designed to enable A/B comparison—but that comparison has not been performed.

7.4 Data Coverage and Freshness

The system was indexed from a single crawl capped at 200 pages under the default configuration. This represents a small fraction of the full SUSTech website, which spans thousands of pages. Pages that require authentication, are rendered dynamically by JavaScript, or were simply not reached within the crawl's BFS depth are absent from the index. A user querying for information on such pages will either receive a hallucinated answer (if the language model draws on parametric knowledge without self-RAG flagging it) or an explicit acknowledgement that the information is unavailable.

Campus information is also temporally perishable. Course offerings, administrative contacts, office locations, and event announcements all change on short timescales. The current system has no incremental update mechanism; re-indexing requires a full re-crawl. A production deployment would require a scheduled re-crawl strategy with change detection to avoid re-processing unchanged pages and to surface outdated indexed content for removal.

7.5 Future Directions

In priority order, the most impactful improvements would be:

1. **Evaluation dataset.** Construct a benchmark of 100–200 (question, answer, relevant passages) triples spanning the range of campus information needs: procedural questions, factual lookups, event queries, and out-of-scope control questions. This is a prerequisite for all subsequent empirical work.
2. **Ablation study.** Measure the isolated contribution of sparse retrieval, RRF fusion, cross-encoder reranking, and self-RAG reflection. Identify which components justify their complexity and latency cost on this specific corpus.
3. **Incremental indexing.** Implement a scheduled re-crawl with page-level change detection (e.g., content hash comparison) to keep the knowledge base current without full re-indexing.
4. **Broader crawl coverage.** Extend the crawl to JavaScript-rendered content via a headless browser, and—with appropriate access permissions—authenticated sections of the website.
5. **Structured data integration.** Campus data in structured formats (course timetables, exam schedules, room availability) is better queried through structured interfaces than through free-text RAG. A hybrid router that dispatches structured queries to a database backend and unstructured queries to the RAG pipeline would serve a wider range of campus information needs.

8 Conclusion

This project demonstrates that a practical, local-first RAG system for campus information retrieval can be built entirely from open-weight models and open-source infrastructure, without relying on external API services. The system covers the complete pipeline from web crawling through text preprocessing, semantic chunking, hybrid retrieval, cross-encoder reranking, self-RAG reflection, and streaming generation—deployed on hardware ranging from a single CPU laptop to a three-GPU server.

Several design decisions are worth highlighting. The paragraph-first chunking strategy with structured-page detection preserves the coherence of short announcement-style content that would be mangled by naive fixed-window splitting. The hybrid retrieval architecture addresses a known weakness of dense-only retrieval on exact-match queries, which are common in the campus domain (specific course codes, building names, contact details). The self-RAG loop introduces explicit scope and factual-support checks that prevent the model from generating confident but ungrounded responses when the indexed corpus does not contain the answer. The relay-worker deployment decouples API availability from GPU hardware placement, enabling flexible distributed operation without requiring the GPU machine to be publicly addressable.

The most important gap identified by this work is the absence of quantitative evaluation. The design choices enumerated above are motivated by prior literature and engineering reasoning, but their actual effect on this system’s answer quality over this corpus has not been measured. As detailed in Chapter 7, constructing a labelled benchmark for the SUSTech domain and using it to run ablations across retrieval modes, chunk sizes, and self-RAG configurations is the highest-priority next step. Without such evaluation, the system is a well-engineered prototype rather than a validated tool.

Subject to that caveat, the system is functional and deployable. It indexes SUSTech public web content, answers in-scope queries with source citations, explicitly acknowledges out-of-scope or insufficiently evidenced questions, and streams responses in real time to a responsive web interface.

References

- Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024). “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *International Conference on Learning Representations*.
- Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., and Liu, Z. (2024). “C-Pack: Packaged Resources to Advance General Chinese Embedding”. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 641–649.
- Chroma Team (2023). *Chroma: The Open-Source Embedding Database*. (Accessed June 2025). Available from: <https://www.trychroma.com>.
- Cormack, G. V., Clarke, C. L. A., and Buettcher, S. (2009). “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 758–759.
- Gerganov, G. (2023). *llama.cpp: LLM Inference in C/C++*. (Accessed June 2025). Available from: <https://github.com/ggerganov/llama.cpp>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pp. 611–626.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. 33, pp. 9459–9474.
- Qwen Team (2025). Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*.
- Reimers, N. and Gurevych, I. (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 3982–3992.
- Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*. 3.4, pp. 333–389.